



Früherkennung von Diabetes mittels Machine Learning Algorithmen

Gruppe 3

Heyne Ruben, Matnr.: 3157992
Mathan Denis, Matnr.: 3488601
Kitzmann Paul, Matnr.: 3434752
Schießl Joseph, Matnr.: 3431384

Studiengang: Informatik B. Sc.

Semester: IN3/4, Sommersemester 2025

Datum: 3. August 2026

Ostbayerische Technische Hochschule Regensburg

Modul: Einführung in Machine Learning

Dozent: Prof. Dr. Brijnesh Jain

Inhaltsverzeichnis

1. Einleitung	1
2. Verwandte Arbeiten	2
3. Methode	2
4. Experimente	4
4.1. Daten	4
4.2. Setup	5
4.3. Ergebnisse und Diskussion	6
5. Fazit	10
A. Anhang	13
A.1. MLP	13
A.2. Hyperparameter Scikit-Learn	16
A.3. Zusätzliche Plots, Tabellen	19

1. Einleitung

Diabetes wird in unserer Gesellschaft zunehmend zu einem großen Problem. In Deutschland leben aktuell etwa 11 Millionen Menschen mit Diabetes (Prävalenz), und jährlich erkranken über eine halbe Million Erwachsene neu (Inzidenz).[1] Dabei handelt es sich überwiegend um Typ-2-Diabetes, die häufigste Form, die meist im Erwachsenenalter auftritt und eng mit Lebensstilfaktoren zusammenhängt. Daneben gibt es Typ-1-Diabetes, eine Autoimmunerkrankung, die meist bereits im Kindes- oder Jugendalter beginnt, sowie Schwangerschaftsdiabetes, das vorübergehend während der Schwangerschaft auftritt, aber ebenfalls langfristige Risiken birgt.[11] Bei gleichbleibender Entwicklung könnten bis 2040 rund 12,3 Millionen Betroffene erwartet werden. Vor der Diagnose leben viele Menschen etwa acht Jahre mit unentdecktem Diabetes. Die Dunkelziffer wird auf etwa 2 Millionen geschätzt [1].

Mit diesen Zahlen liegt Deutschland hinsichtlich der Diabetes-Prävalenz im europäischen Vergleich im Mittelfeld. Weltweit betrachtet lässt sich auch eine steigende Prävalenz beobachten. Während es 2021 noch 537 Millionen bekannte Fälle gab, gehen Experten davon aus, dass es bis 2045 783 Millionen Menschen mit diagnostiziertem Diabetes geben wird, wie in Abbildung 1 zu sehen ist. Auffallend dabei ist, dass die Prävalenz in Ländern mit niedrigem bis mittleren Einkommen am höchsten ist. Das sind vermutlich Länder mit schlechterer medizinischen Versorgung, wodurch die Dunkelziffer noch viel höher sein könnte als in Deutschland.[11]

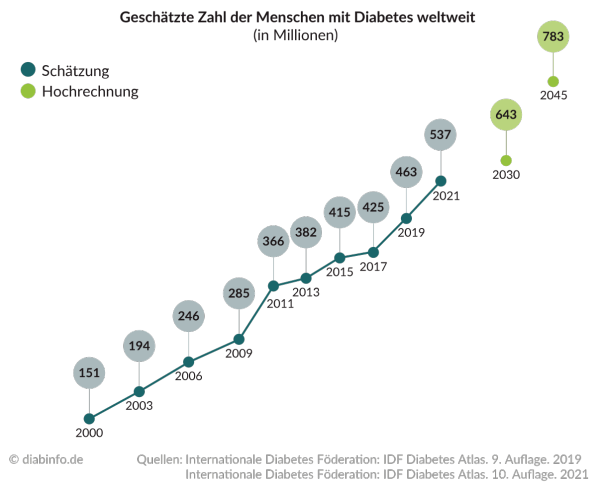


Abbildung 1: Geschätzte Zahl der Menschen mit Diabetes weltweit[11]

Eine frühzeitige Diagnose ist oft schon zu spät, denn zuvor sind Patienten im Stadium Prädiabetes, in dem die Blutzuckerwerte erhöht, aber laut Experten noch nicht diabetisch sind. In dieser Phase lässt sich durch eine Änderung des Lebensstils die Entstehung von Diabetes noch verhindern. Wird dieses Zeitfenster verpasst, ist es umso wichtiger, Diabetes so früh wie möglich zu erkennen und zu behandeln. Hohe Blutzuckerwerte können langfristig die Blutgefäße schädigen und dadurch verschiedene Organe beeinträchtigen. Besonders häufig treten Herz-Kreislauf-Erkrankungen sowie Schäden an Nieren, Augen und Nerven auf. Zudem sind auch andere Erkrankungen wie Krebs oder neurologische Störungen bei Diabetes vermehrt zu beobachten.[11]

2. Verwandte Arbeiten

Bereits heute existieren eine Vielzahl erprobter Machine-Learning-Modelle[6], die sich in der frühzeitigen Erkennung von Diabetes als wirkungsvoll erwiesen haben – darunter bewährte Methoden wie die Logistic Regression, Support Vector Machines und neuronale Netzwerke.

Beispielsweise wird in einer Studie aus dem Jahr 2023, die auf einem unausgeglichene Patientendatensatz aus dem Irak basiert, ein neuartiges, pipelinebasiertes Multi-Klassen-Klassifikationsframework vorgestellt, das auf der Ensemble-Methode basiert. Dabei werden mehrere Machine-Learning-Modelle zu einem Gesamtmodell zusammengeführt, um die Vorhersagegenauigkeit durch kollektive Entscheidungsfindung zu verbessern. Diese Methode scheint mit einer Precision von 0.9861 bei einem Recall von 0.9792 sehr gut zu funktionieren. Ein Nachteil dieses Systems liegt im Aufwand der Entwicklung der vielen verschiedenen Modelle, um ein gutes gemeinsames Ensemble zu entwickeln.[2]

In einer weiteren Studie haben Forscher zwei Modelle entwickelt, um das Risiko für Typ-2-Diabetes vorherzusagen – eines ohne Bluttests und eines mit vier Blutwerten. Grundsätzlich haben sie dafür die gängigen Machine-Learning-Modelle auf die Daten angewandt. Das Modell mit den Blutwerten hat besser abgeschnitten als das andere.[9]

Unser Ziel ist es, Personen mit erhöhtem Diabetes-Risiko frühzeitig zu erkennen, um rechtzeitig präventive Maßnahmen und Diagnostik zu ermöglichen. Dafür entwickeln wir einen neuen Machine-Learning-Algorithmus, der komplexe Datenmuster in angemessener Zeit erfassen und präzise Vorhersagen geben kann.

3. Methode

Als neue Methode stellen wir das **Gradient Boosting** vor.

Zuerst werden alle in diesem Kapitel vorkommenden Variablen deklariert:

Symbol	Beschreibung
$n \in \mathbb{N}$	Anzahl der Trainingsdaten
$\{(x_i, y_i)\}_{i=1}^n$	Training Set
x_i	i-ter Featurevektor
y_i	i-tes Label
$\alpha \in \mathbb{R}$	Gewicht
$h: x_i \rightarrow \mathbb{R}$	Estimator
$F: x_i \rightarrow \mathbb{R}$	Vorhersagefunktion
$L: (y_i, \hat{y}_i) \rightarrow \mathbb{R}$	Lossfunktion
$M \in \mathbb{N}$	Anzahl der Iterationen

Der Algorithmus für **Gradient Boosting** lautet:

0. Input: Training Set $\{(x_i, y_i)\}_{i=1}^n$, differenzierbare Lossfunktion L , Anzahl der Iterationen M .
1. Initialisiere $F_0(x_i)$ mit einem konstanten Wert, z.B. $\frac{1}{n} \sum_{i=1}^n y_i$
2. Von $m = 1$ bis M :
 - a) Berechne die *Pseudo Residuals* $r_{im} = -\frac{\partial L(y_i, F_{m-1}(x_i))}{\partial F_{m-1}(x_i)}$
 - b) Trainiere $h_m(x)$ mit dem Training Set $\{(x_i, r_{im})\}_{i=1}^n$
 - c) Berechne $\alpha_m = \arg \min_{\alpha} \sum_{i=1}^n L(y_i, F_{m-1}(x_i) + \alpha h_m(x_i))$
 - d) Update das Modell: $F_m(x) = F_{m-1}(x) + \alpha_m h_m(x)$
3. Gebe $F_M(x)$ aus.

Gradient Boosting basiert auf *Boosting*. Bevor auf den Hauptalgorithmus eingegangen wird, wird dieses Konzept kurz erläutert. Boosting ist eine *Ensemble* Methode, die auf Basis der gewichteten Mehrheitsentscheidung von $m \in \mathbb{N}$ schwachen Estimatoren $h_1, h_2, \dots, h_m$ entscheidet. Beim *Gradient Boosting* werden als Estimatoren einfache Entscheidungsbäume verwendet, die keine eindeutigen Labels, sondern sogenannte reelle *Logits* zurückgeben. Die gewichtete Vorhersagefunktion F ergibt sich dann mithilfe der Gewichte $\alpha_1, \alpha_2, \dots, \alpha_m \in \mathbb{R}$:

$$F(x_i) = \sum_{i=1}^m \alpha_i h_i(x_i)$$

Obwohl die einzelnen Estimatoren nur eine etwas bessere Accuracy als bloßes Raten haben, erzeugt Boosting sehr leistungsstarke Modelle. Die zentrale Herausforderung besteht darin, geeignete Gewichte und Estimatoren zu finden.

Gehen wir davon aus, dass der Algorithmus M Iterationen hat und in jeder Iteration m ($1 \leq m \leq M$) eine fehlerhafte Vorhersagefunktion $F_{m-1}(x)$ vorliegt. Dann kann $F_m(x)$ durch einen gut gewählten Estimator $h_m(x)$ und einem Gewicht $\alpha_m \in \mathbb{R}$ verbessert werden.

$$F_m(x_i) = F_{m-1}(x_i) + \alpha_m h_m(x_i)$$

Gradient Boosting bestimmt hierfür $h_m(x_i)$ so, dass dieser in Richtung des **steilsten Abstiegs** der Lossfunktion zeigt. Diese muss daher differenzierbar sein. Hierfür werden sogenannte *Pseudo Residuals* $r_{1m}, r_{2m}, \dots, r_{nm}$ mit Hilfe von *gradient descent* berechnet:

$$r_{im} = -\frac{\partial L(y_i, F_{m-1}(x_i))}{\partial F_{m-1}(x_i)}$$

Anschließend wird $h_m(x)$ mit dem Training Set $\{(x_i, r_{im})\}_{i=1}^n$ trainiert. Durch die Eigenschaften von r_{im} erhält man ein $h_m(x)$, das den Loss der aktuellen Vorhersagefunktion

$F_{m-1}(x)$ verringert. Die optimale Schrittweite α_m wird berechnet, indem folgendes Optimierungsproblem gelöst wird:

$$\alpha_m = \arg \min_{\alpha} \sum_{i=1}^n L(y_i, F_{m-1}(x_i) + \alpha h_m(x_i))$$

In der Theorie kann $F(x_i)$ somit beliebig oft verbessert werden. Der Startpunkt $F_0(x_i)$ wird auf einen beliebigen konstanter Wert, zum Beispiel das arithmetische Mittel, gesetzt:

$$F_0(x_i) = \frac{1}{n} \sum_{i=1}^n y_i$$

4. Experimente

Im folgenden Teil wenden wir unser Verfahren und die Bestehenden auf verschiedene Datensätze an und definieren Bewertungskriterien, um eine objektive Grundlage für die anschließende Diskussion zu schaffen.

4.1. Daten

Wir verwenden vier verschiedene Datensätze mit unterschiedlichen Features, die sich teilweise überschneiden. Häufig verwendete Features sind BMI, Glucose, Cholesterin und das binäre Feature Diabetes, das zur Klassifikation dient.

Der erste[4] und zweite[17] Datensatz müssen nicht angepasst werden. Beim ersten liegt die Besonderheit darin, dass er fast ausschließlich kategorische Merkmale enthält. So entspricht beispielsweise der Wert 1 im Feature age dem Altersbereich 18 - 24 Jahre. Darüber hinaus weist dieser im Vergleich zu den anderen Datensätzen deutlich mehr binäre Variablen auf. Für den zweiten Datensatz empfiehlt sich aufgrund seiner kleinen Größe eine Cross Validation. Spalten, die textuelle oder numerische Encodings enthalten, werden folgendermaßen One-Hot-Encoded[16]:

- 3. Datensatz[14]:
 - gender('Female', 'Male', 'Other') -> Female, Male
 - smoking('neverSm', 'formerSm', 'currentSm', 'nCurrentSm', 'noInfo')
-> neverSm, formerSm, currentSm, nCurrentSm
- 4. Datensatz[5]:
 - smoking(1, 2, 3) -> smoking_1, smoking_2, smoking_3
 - drinking(1, 2, 3) -> drinking_1, drinking_2, drinking_3

Außerdem besitzt der vierte Datensatz [5] eine Besonderheit: In mehreren Feldern kommt ausschließlich bei Diabetikern der Wert 4.860753 vor. Um zu verhindern, dass unsere Modelle diesen Wert auswendig lernen, entfernen wir die betroffenen Zeilen (Ergibt 4* in Tabelle 1).

Nr	Zeilenanzahl	Spaltenanzahl	Anteil Positives
1	253680	21	13.9%
2	888	9	34.9%
3	100000	9	8.5%
4	4303	18	30.3%
4*	3260	18	8.0%

Tabelle 1: Verwendete Datensätze

4.2. Setup

Als Auswahlkriterium der Modelle wird ein Recall, auch bekannt als die Sensitivität, von mindestens 80% vereinbart. Der Wert von 80% hat keine wissenschaftliche Relevanz und ist von uns innerhalb des Projekts frei gewählt. Von den Modellen mit einem ($Recall \geq 80\%$) wählen wir das mit der höchsten Precision aus. Bei medizinischen Themen ist die Sensitivität ausschlaggebend, um mehr Personen mit einer Krankheit korrekt zu klassifizieren, auf Kosten von mehr False-Positives. Dadurch wird gewährleistet, dass kranke Personen erkannt werden und danach für eine Behandlung oder weitere Untersuchungen angemeldet werden können. Es wäre fatal eine kranke Person zu entlassen, aufgrund einer Fehlklassifikation, ganz nach dem Motto "Vorsicht ist besser als Nachsicht".

Die Hyperparametersuche wird für alle Datensätze standardisiert durchgeführt, jedoch hat jedes Modell seine eigenen möglichen Hyperparameter. So werden für die Modelle Logistic Regression, Random Forest und Gradient Boosting eine Mischform der Grid- und Random Search Methodik durchgeführt. Die Hyperparametersuche bei Multi-Layer Perceptron (MLP) wird im Hintergrund durch eine Form der Random Search Methodik realisiert. Hierfür wird eine Python Bibliothek namens RayTune[13] verwendet.

Die verwendeten und letztendlich ausgewählten Hyperparameter sind im Anhang zu finden (Vgl. Abschnitt A.2). In den folgenden Unterpunkten werden Besonderheiten in den jeweiligen Modellen erläutert.

Multi-Layer Perceptron

Die Klasse der Multi-Layer Perceptron (MLP), auch bekannt als Neuronale Netze, ist sehr leistungstark. Die Implementierung erfolgt hauptsächlich mit den Python Bibliotheken PyTorch[3] (Framework für Deep Learning) und RayTune[13] (Framework für Hyperparametersuche). Als Optimizer für alle Datensätze wird ADAM[12] gewählt und die Epochenanzahl wird auf 100 festgelegt. Als Scheduler für das Training wird der ASHA Scheduler von RayTune[13] ausgewählt. Eine Tabelle mit allen getesteten Werten ist im Anhang zu finden (Vgl. Tabelle 3).

SVM

Die Experimente mit den Support Vector Machines (SVMs) werden mithilfe der Python-Bibliothek Scikit-Learn[15] umgesetzt. Getestet werden die Kerntypen linear, polynomial, radial basis function (RBF) und sigmoid. Für jeden Kerntyp wird zunächst eine separate Hyperparameteroptimierung durchgeführt (Vgl. Tabelle 7). Anschließend werden die jeweils leistungsstärksten Modelle untereinander verglichen.

Aufgrund zeitlicher Einschränkungen wird während der Hyperparameteroptimierung der Toleranzwert tol , welcher die Abbruchbedingung des Trainings steuert, vorübergehend vom Standardwert 10^{-3} auf 1.0 erhöht, um die Rechenzeit deutlich zu verkürzen. Wir sind uns bewusst, dass durch diese Maßnahme potenziell optimalere Modelle übersehen werden können. Dennoch liefert dieser Ansatz die Ergebnisse, die unserem Ziel der angemessenen Zeit entsprechen um eine fundierte Vergleichbarkeit mit dem eigenen Gradient-Boosting-Verfahren zu gewährleisten. Das finale Modell hat dann die ermittelte Hyperparameterkonfiguration, jedoch unter Verwendung des Standardwerts $\text{tol}=10^{-3}$. Darüber hinaus wird in den beiden größeren Datensätzen mit einer deutlich kleineren Trainingsmenge von 0.05 gearbeitet.

Gradient Boosting

Gradient Boosting wird mit Scikit-Learn implementiert. Für die großen Datensätze 1 und 3 wird aus Effizienzgründen *HistGradientBoostingClassifier*[8] benutzt. Bei den Datensätzen 2 und 4 wird *GradientBoostingClassifier*[7] verwendet. Die beiden Modelle unterscheiden neben der Rechenzeit auch in den notwendigen Hyperparametern (Vgl. Tabelle 10 und 11).

4.3. Ergebnisse und Diskussion

MLP

Die Hyperparameter werden mittels RayTune[13] auf dem Validation Set ermittelt.

Pro Datensatz werden sehr unterschiedliche Ergebnisse der Hyperparameter berechnet (Vgl. Tabelle 4).

Die im Anhang beigefügten Grafiken zeigen die jeweils fünf besten MLP Modelle für die vier Datensätze über die Epochen hinweg. Einige der Graphen enden frühzeitig nach 25 oder 75 Epochen (Vgl. Abbildung 5a - 8d). Dies ist dem Early-Stopping des ASHA Schedulers von RayTune[13] zu zuschreiben, welcher das Training von Modellen frühzeitig abbricht, sollte keine nennenswerte Verbesserung stattfinden. Alle Modelle fallen sehr schnell im Recall ab und pendeln sich meist bei circa 55-60% ein. Hingegen steigt der F1-Score zunehmend an, da die Precision (nicht explizit mit aufgeführt) weiter ansteigt. Interessanterweise werden auf allen Datensätzen gute Accuracy Werte erreicht. Diese liegen meist im Bereich von circa 80-96% Accuracy auf dem Validation Set.

Dennoch werden alle Modelle nach dem festgelegten Kriterium ($\text{Recall} \geq 80\%$) aussortiert, da nach dem Training keines der Modelle das Kriterium auf dem Validation Set erfüllt. Ein Modell zu wählen, welches nur einige Epochen trainiert wird, ist nicht

sinnvoll, da alle anderen Metriken sich kaum entwickeln können.

Das beste Modell pro Datensatz hat folgende Werte:

Datensatz	Precision	Recall	F1 Score	Validation Accuracy	Test Accuracy
1	49.00%	24.00%	32%	84.49%	84.45%
2	64.00%	78.00%	71%	77.27%	68.10%
3	100.00%	68.00%	81%	97.02%	97.07%
4	73.00%	50.00%	59%	94.60%	94.68%

Tabelle 2: Metriken des besten Modells pro Datensatz

Aufgrund von Hardware Limitationen und zeitlicher Einschränkung innerhalb des Projekts, liefern MLPs bei allen Datensätzen eher unattraktive Ergebnisse, im Bezug auf einen produktiven Einsatz.

Vergleicht man die Ergebnisse mit einer aktuellen Studie aus dem Jahr 2024, in welcher es um die Früherkennung von Diabetes Typ-1 geht, liegt unser Modell weit hinter dem Recall Wert der Studie von 99% und einer Accuracy von 99,8%. [10]

Man kann annehmen, dass durch weitere Verbesserungen der Architektur und Erweiterung des Suchraumes der Hyperparameter für das Modell bessere Ergebnisse erzielt werden können. Aufgrund der bereits erwähnten begrenzten Ressourcen und des engen Zeitrahmens müssen jedoch diese Verbesserungen und weitere Experimente auf zukünftige Arbeiten verschoben.

Grafischer Vergleich

Damit die Modelle mit den besten Hyperparameterkonfigurationen miteinander verglichen werden können, werden diese graphisch aufbereitet. Zwar unterscheiden sich die Graphiken der verschiedenen Datensätze gravierend in den Werten, aber es sind trotzdem einige Trends zu beobachten. Leider können die Ergebnisse vom MLP nicht auf dieselbe Art und Weise dargestellt werden, wie es mit Scikit-Learn möglich ist und werden in diesem Kapitel nicht betrachtet (Für die Ergebnisse von MLP vgl. Abschnitt 4.3).

Zur Erinnerung: Wir suchen das Modell mit der höchsten Precision bei einem Recall von 80%. Die folgenden Aussagen beziehen sich auf Ergebnisse der verwendeten Datensätze (Vgl. Abb. 2, 3, 4, 9 - 11).

In den meisten Fällen ist Gradient Boosting am besten - besonders bei Datensätzen mit vielen Elementen. Random Forest erzielt zwar bessere Ergebnisse in den Trainingsmengen (Vgl. Abb. 12 - 15), kann aber nicht so gut wie Gradient Boosting generalisieren. Die SVM kann selten die Logistic Regression übertreffen, obwohl sie vor allem bei großen Datensätzen sehr lange rechnen muss. Dies könnte auf die Heuristik in der Hyperparame-

tersuche zurückgeführt werden. Logistic Regression liefert erwartungsgemäß schlechtere Ergebnisse als die Ensemble-Methoden.

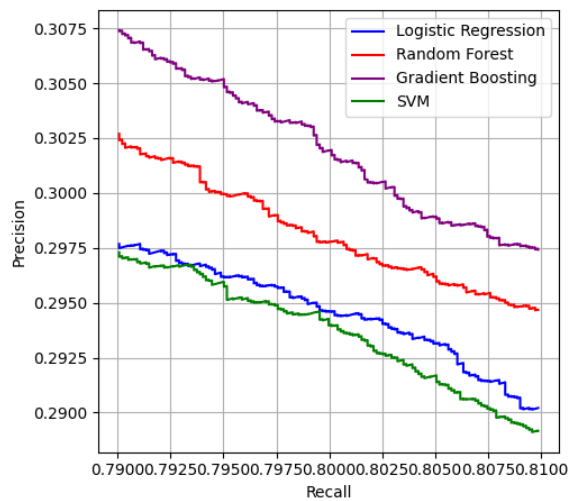


Abbildung 2: Performance Datensatz 1

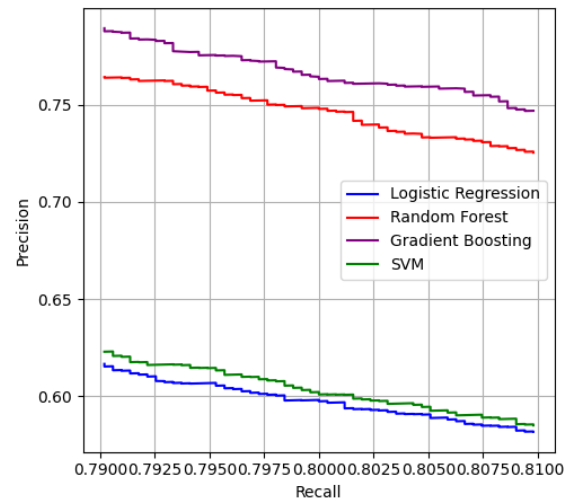


Abbildung 3: Performance Datensatz 3

Bei kleineren Datensätzen ist es schwierig, Aussagen im Bezug auf die Gesamtperformance zu treffen, weil diese sehr von ihrer Trainings- und Testmenge abhängen. Es entsteht trotz hoher Varianz beim vierten Datensatz die Tendenz, dass Gradient Boosting die höchste Precision haben könnte. Dies ist allerdings nur bei einem Recall von 80% der Fall. Bei anderen Recall-Regionen entsteht ein äußerst chaotischer Eindruck (Vgl. Abb. 4).

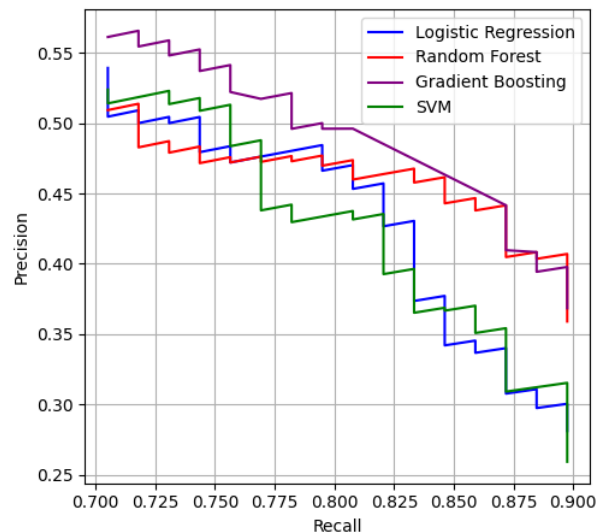
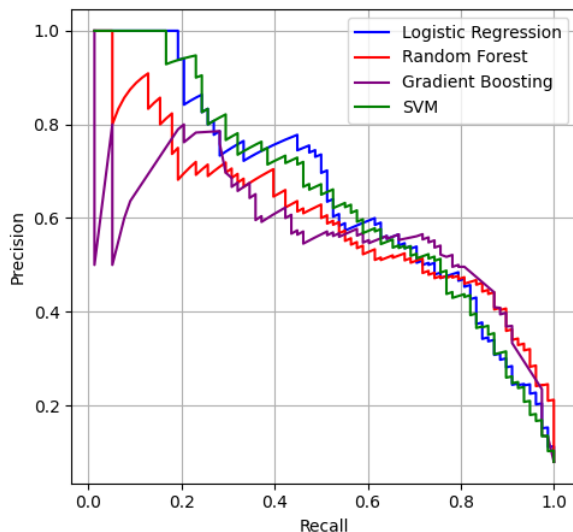


Abbildung 4: Performance Datensatz 4

Bemerkenswert ist, dass sich Datensatz 1 mit einer geringen Precision von ca. 30% deutlich von den anderen unterscheidet, obwohl mit Abstand am meisten Daten vorhanden

sind (Vgl. Tabelle 1). Ein Grund könnte sein, dass er keine numerischen, sondern (fast) nur binäre Features hat. Mit diesen können vielleicht einfach keine guten Ergebnisse erzielt werden. Versucht man, die Bedeutung der einzelnen Spalten durch die Koeffizienten der Logistic Regression abzuschätzen, so besteht bei hohen Blut- und Cholesterinwerten, einem hohen BMI oder bei schlechter Gesundheit ein höheres Risiko für Diabetes (Vgl. Tab. 13).

Es ist zumindest zweifelhaft, ob diese Merkmale auch eine medizinische Rechtfertigung haben, aber sie scheinen auf den ersten Blick relativ intuitiv zu sein. Allerdings muss auch gesagt sein, dass diese Gewichtung der Merkmale zu sehr schwachen Ergebnissen führt (Vgl. Abb. 2). Eine weitere Idee wäre, durch Preprocessing der Daten die Qualität der Ergebnisse zu verbessern. Da aber komplexere Modelle wie Gradient Boosting und Random Forest kaum bessere Ergebnisse als Logistic Regression erzielen können, werden die Ertragschancen von Preprocessing gegenüber von langen Rechenzeiten und Bearbeitungsaufwand als unverhältnismäßig gering eingeschätzt.

Im Gegensatz dazu liefert Datensatz 3 äußerst starke Ergebnisse (Vgl. Abb. 3). Bei einem Recall von 80% erreicht Gradient Boosting eine Precision von ca. 77%. Das bedeutet, dass eine positive Diagnose zu 77% Diabetes vorhersagt, und dabei nur jeder fünfte Fall untergeht. Damit entstehen seltener falsch positive Diagnosen.

Erstaunlich ist hierbei, dass die Precision von Gradient Boosting die Werte von Random Forest (-2%) und SVM (-17%) deutlich übertrifft (Vgl. Abb. 3). Leider können wir nicht einsehen, welche Features im Datensatz 3 Gradient Boosting so stark machen, aber im Vergleich zu den anderen Datensätzen sind folgende Eigenschaften einzigartig:

- Großer Datensatz mit wenig Features
- Feature mit HbA1c-Level (durchschnittlicher Blutzuckerspiegel der letzten 8 bis 12 Wochen)
- Genaue Werte bei Alter, BMI, HbA1c und Blutzucker

Trotz der geringen Anzahl an Features erreicht Datensatz 3, wie oben beschrieben, im Vergleich sehr gute Werte. Es scheint auch eine positive Korrelation zwischen dem HbA1c-Level und der Aussagekraft über Diabetes zu geben, weil dieser in keinem anderen Datensatz vorkommt. Zudem scheinen exakte Werte bessere Ergebnisse erzielen zu können als gelabelte oder klassifizierte Features.

Es sei auch gesagt, dass sowohl Random Forest als auch Gradient Boosting beim zweiten Datensatz schlechter als Logistic Regression und SVM sind (Vgl. Abb. 10). Da sehr viele Werte im zweiten Datensatz fehlen, ist es möglich, dass dadurch die Qualität der Entscheidungsbäume zu stark beeinträchtigt worden sein könnte. Es wurde aus zeitgründen und dem schon gut performenden Datensatz 3 darauf verzichtet, die Ergebnisse durch Preprocessing zu verbessern.

5. Fazit

In dieser Arbeit wurde untersucht, inwiefern sich verschiedene Machine-Learning Algorithmen zur Früherkennung von Diabetes eignen. Ziel war es, bei einem Recall von mindestens 80 % die jeweils beste Modellvariante hinsichtlich der Precision zu identifizieren. Die Analyse auf vier verschiedenen Datensätzen zeigte, dass insbesondere Gradient Boosting zuverlässige Ergebnisse liefert, während einfache Modelle wie die Logistic Regression erwartungsgemäß schlechter abschnitten. Die MLP-Modelle konnten trotz hoher Accuracy-Werte die Recall-Vorgabe nicht erfüllen und sind in ihrer aktuellen Form für eine klinische Anwendung nicht geeignet.

Die Qualität der Ergebnisse variierte stark in Abhängigkeit von den verwendeten Daten. Datensätze mit klaren, numerischen Merkmalen wie dem HbA1c-Level führten zu besseren Vorhersagen als solche mit rein kategorischen oder binären Features. Dies unterstreicht die zentrale Bedeutung einer hochwertigen Datenbasis für medizinisches Machine Learning.

Limitationen dieser Arbeit liegen in den begrenzten Ressourcen, die insbesondere das Training tiefer neuronaler Netze einschränkten. Zudem wurde das Feature Engineering nur rudimentär berücksichtigt. Eine ausführliche Modellinterpretation – insbesondere für Gradient Boosting – steht ebenfalls noch aus.

Zukünftige Arbeiten sollten sich auf die Erweiterung des Hyperparameterraums, die Kombination mehrerer Modelle (z.B. durch Ensembles) sowie auf die Integration weiterer medizinisch relevanter Features fokussieren. Zudem wäre eine engere Zusammenarbeit mit medizinischen Expertinnen und Experten sinnvoll, um die Modellbewertung praxisnäher und ethisch fundierter gestalten zu können.

Literatur & Datensätze

- [1] diabetesDE – Deutsche Diabetes-Hilfe e.V. URL: https://www.diabetesde.org/ueber_diabetes/was_ist_diabetes_/diabetes_in_zahlen (besucht am 27.06.2025).
- [2] K. Abnoosian, R. Farnoosh und M.H. Behzadi. „Prediction of diabetes disease using an ensemble of machine learning multi-classifier models“. In: *BMC Bioinformatics* 24 (2023), S. 337. DOI: [10.1186/s12859-023-05465-z](https://doi.org/10.1186/s12859-023-05465-z). URL: <https://doi.org/10.1186/s12859-023-05465-z>.
- [3] Jason Ansel u. a. „PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation“. In: *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. ACM, Apr. 2024. DOI: [10.1145/3620665.3640366](https://doi.org/10.1145/3620665.3640366). URL: <https://docs.pytorch.org/assets/pytorch2-2.pdf>.
- [4] *CDC Diabetes Health Indicators*. URL: <https://archive.ics.uci.edu/dataset/891/cdc+diabetes+health+indicators> (besucht am 30.06.2025).
- [5] Parisa Karimi Darabi. *Diabetes Dataset With 18 Features*. URL: <https://www.kaggle.com/datasets/pkdarabi/diabetes-dataset-with-18-features> (besucht am 30.06.2025).
- [6] Databricks Inc. *Machine-Learning-Modelle*. <https://www.databricks.com/de/glossary/machine-learning-models>. 160 Spear Street, 15th Floor, San Francisco, CA 94105, USA. (Besucht am 02.07.2025).
- [7] Scikit-learn developers. *sklearn.ensemble.GradientBoostingClassifier*. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingClassifier.html>. Accessed: 2025-06-30. 2024.
- [8] Scikit-learn developers. *sklearn.ensemble.HistGradientBoostingClassifier*. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingClassifier.html>. Accessed: 2025-06-30. 2024.
- [9] Y. Edlitz und E. Segal. „Prediction of type 2 diabetes mellitus onset using logistic regression-based scorecards“. In: *eLife* 11 (2022), e71862. DOI: [10.7554/eLife.71862](https://doi.org/10.7554/eLife.71862). URL: <https://doi.org/10.7554/eLife.71862>.
- [10] Sandeep Ghosh, Sudipta Mandal und Arun Kumar Chatterjee. „Predicting diabetes using a multilayer perceptron neural network: an ensemble approach“. In: *Scientific Reports* 14.1 (2024). Zugriff am 29. Juni 2025, S. 5013. DOI: [10.1038/s41598-024-51438-4](https://doi.org/10.1038/s41598-024-51438-4). URL: <https://www.nature.com/articles/s41598-024-51438-4>.
- [11] Helmholtz Zentrum München, Deutsches Zentrum für Diabetesforschung und Deutsches Diabetes-Zentrum. *Diabetes in Deutschland – Zahlen und Fakten*. URL: <https://www.diabinfo.de/zahlen-und-fakten.html> (besucht am 27.06.2025).
- [12] Diederik P. Kingma und Jimmy Ba. *Adam: A Method for Stochastic Optimization*. 2017. arXiv: [1412.6980](https://arxiv.org/abs/1412.6980) [cs.LG]. URL: <https://arxiv.org/abs/1412.6980>.

- [13] Richard Liaw u. a. „Tune: A Research Platform for Distributed Model Selection and Training“. In: *arXiv preprint arXiv:1807.05118* (2018).
- [14] Mohammed Mustafa. *Diabetes prediction dataset*. URL: <https://www.kaggle.com/datasets/iammustafatz/diabetes-prediction-dataset/data/> (besucht am 30.06.2025).
- [15] F. Pedregosa u. a. „Scikitlearn: Machine Learning in Python“. In: *Journal of Machine Learning Research* 12 (2011), S. 2825–2830.
- [16] Dr Ana Rojo-Echeburúa. *Was ist One Hot Encoding und wie man es in Python implementiert*. URL: <https://www.datacamp.com/de/tutorial/one-hot-encoding-python-tutorial> (besucht am 02.07.2025).
- [17] Saurabh Singh. *diabetes.csv*. URL: <https://www.kaggle.com/datasets/saurabh00007/diabetescsv/data> (besucht am 30.06.2025).

A. Anhang

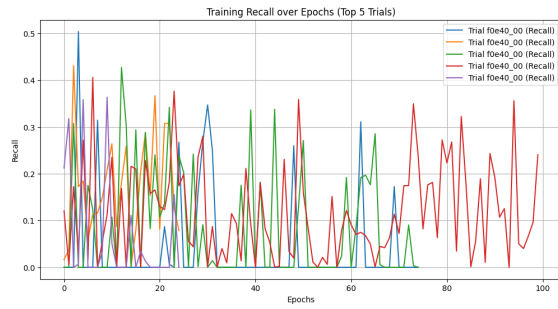
A.1. MLP

Hyperparameter	Beschreibung	Mögliche Werte
Hidden Layers	Tiefe und Anzahl der Input-Neuronen	[32, 16], [64, 32], [64, 32, 16], [128, 64, 32], [256, 128, 64]
Dropout Rate	Prozentsatz deaktivierter Neuronen	Zufällig aus Intervall [0.0, 0.5]
Aktivierungsfunktion	Aktivierungsfunktion der Schichten	Zufällig ausgewählt ReLU oder TanH
Learning Rate	Schrittweite pro Iteration	Zufällig aus Intervall [1e-4, 1e-2]
Weight Decay	Regularisierung der Gewichte	Zufällig aus Intervall [1e-4, 1e-2]
Batch Size	Datenpunkte pro Mini-Batch	8, 16, 32, 64, 128, 256, 512

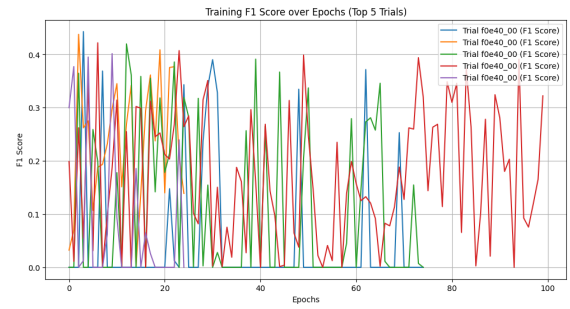
Tabelle 3: Übersicht der Hyperparameter für das MLP Modell

Datensatz	Hidden Layers	Activation	Dropout Rate	Learning Rate	Weight Decay	Batch Size
1	[256, 128, 64]	relu	0.318	0.00138	0.00016	32
2	[32, 16]	tanh	0.022	0.00010	0.00058	512
3	[64, 32, 16]	tanh	0.098	0.00359	0.00501	32
4	[256, 128, 64]	tanh	0.327	0.00475	0.00023	512

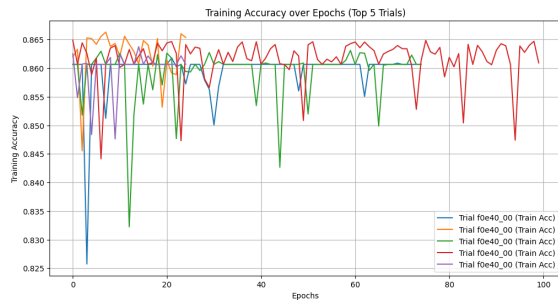
Tabelle 4: Beste Hyperparameterkonfigurationen MLP



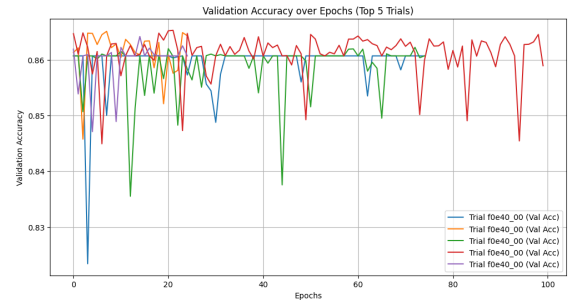
(a) Training Recall



(b) Training F1 Score



(c) Train Accuracy

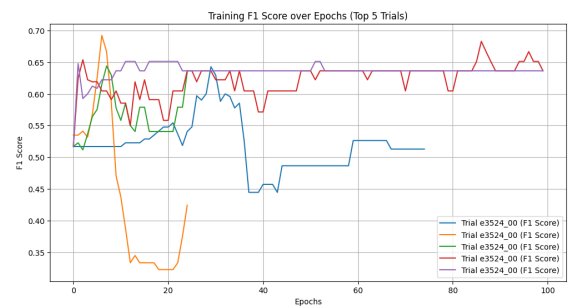


(d) Validation Accuracy

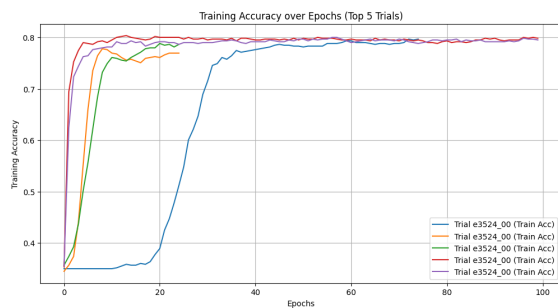
Abbildung 5: MLP Metriken Datensatz 1



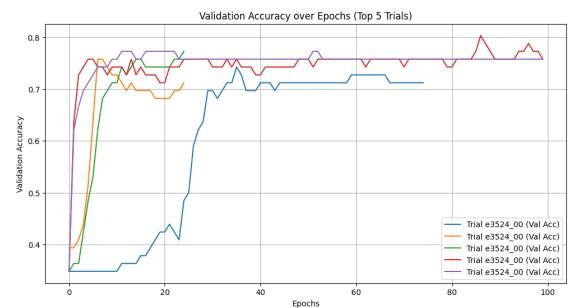
(a) Training Recall



(b) Training F1 Score

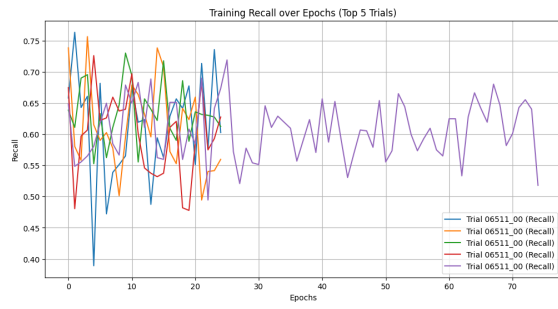


(c) Train Accuracy

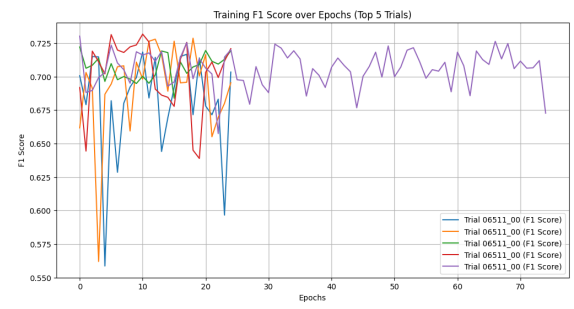


(d) Validation Accuracy

Abbildung 6: MLP Metriken Datensatz 2



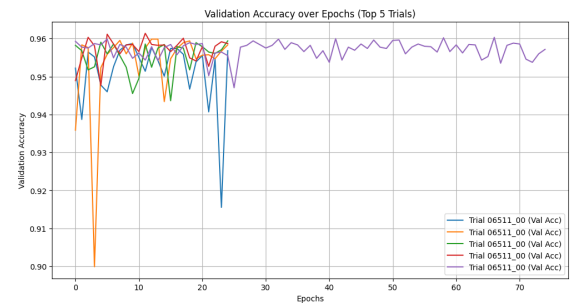
(a) Training Recall



(b) Training F1 Score

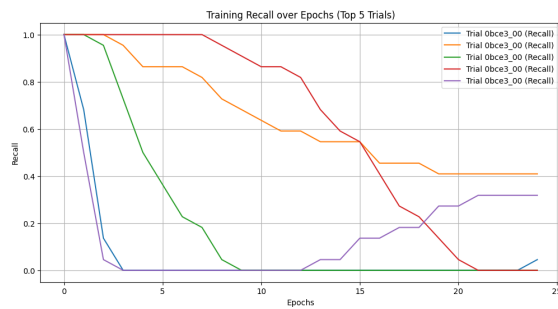


(c) Train Accuracy

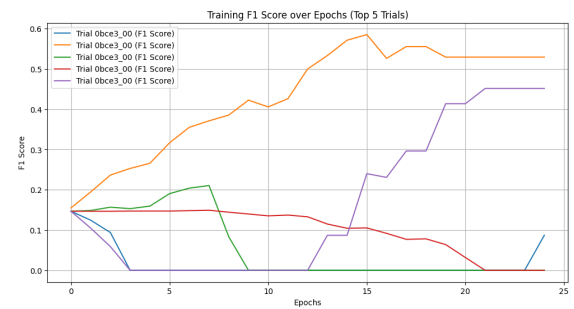


(d) Validation Accuracy

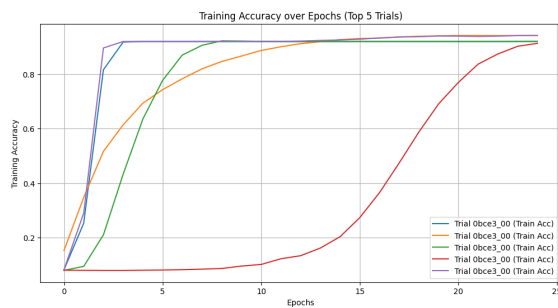
Abbildung 7: MLP Metriken Datensatz 3



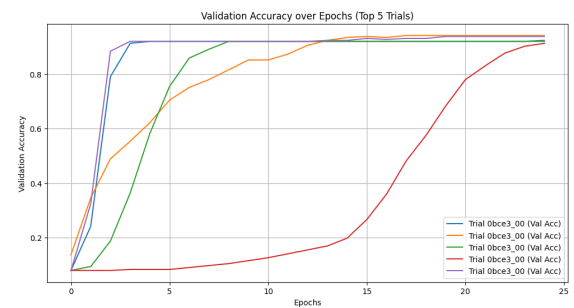
(a) Training Recall



(b) Training F1 Score



(c) Train Accuracy



(d) Validation Accuracy

Abbildung 8: MLP Metriken Datensatz 4

A.2. Hyperparameter Scikit-Learn

Logistic Regression

Hyperparameter	Mögliche Werte
Penalty	l1, l2
C	$2^{-10}, 2^{-9}, 2^{-8}, \dots, 2^7$
Solver	liblinear, saga

Tabelle 5: Übersicht Hyperparameter für Logistic Regression

Datensatz	Penalty	C	Solver
1	l1	2^{-8}	liblinear
2	l1	2^1	liblinear
3	l1	2^{-7}	liblinear
4	l1	2^{-6}	liblinear

Tabelle 6: Übersicht der besten Hyperparameter für Logistic Regression

SVM

Kernel	Hyperparameter	Werte
Linear	C	$[10^{-5}, 10^{-4}, \dots, 10^2]$
	class_weight	[None, "balanced"]
Polynomial	C	$[10^{-5}, 10^{-4}, \dots, 10^2]$
	class_weight	[None, "balanced"]
	degree	[2, 3, 4]
	coef0	[0.0, 0.1, 0.5, 1.0, 1.5, 2.0]
RBF	C	$[10^{-5}, 10^{-4}, \dots, 10^2]$
	class_weight	[None, "balanced"]
	gamma	$[10^{-5}, 10^{-4}, \dots, 10^2, \text{'scale'}, \text{'auto'}]$
Sigmoid	C	$[10^{-5}, 10^{-4}, \dots, 10^2]$
	class_weight	[None, "balanced"]
	gamma	$[10^{-5}, 10^{-4}, \dots, 10^2, \text{'scale'}, \text{'auto'}]$
	coef0	[0.0, 0.1, 0.5, 1.0, 1.5, 2.0]

Tabelle 7: Hyperparameter für verschiedene SVM-Kernel mit jeweiligen Wertebereichen

Random Forest

Hyperparameter	Beschreibung	Mögliche Werte
n_estimators	Anzahl Bäume	$2^4, 2^5, \dots, 2^{10}$
max_depth	maximale Tiefe	2, 4, 8, 16, 22
min_samples_split	minimale Samples ob Knoten erneut aufgespalten wird	2, 4
min_samples_leaf	minimale Samples ob Knoten zum Blatt zu wird	2, 4, 8

Tabelle 8: Übersicht der Hyperparameter für Random Forest

Datensatz	n_estimators	max_depth	min_samples_split	min_samples_leaf
1	10	8	4	2
2	20	8	4	2
3	128	22	2	10
4	20	8	2	4

Tabelle 9: Übersicht der besten Hyperparameter für Random Forest

Gradient Boosting

Hyperparameter	Beschreibung	Mögliche Werte
loss	Verwendete Lossfunktion	log_loss, exponential
learning_rate	Lernrate	$2^{-10}, 2^{-9}, \dots, 2^0$
n_estimators	Anzahl an Estimatoren	$2^4, 2^5, \dots, 2^{10}$
criterion	Funktion zur Messung der Splitqualität im Entscheidungsbaum	friedman_mse, squared_error

Tabelle 10: Übersicht der Hyperparameter für Gradient Boosting

Hyperparameter	Beschreibung	Mögliche Werte
loss	Verwendete Lossfunktion	log_loss
learning_rate	Lernrate	$2^{-10}, 2^{-9}, \dots, 2^0$
max_leaf_nodes	Maximale Anzahl von Knoten im Entscheidungsbaum	$2^4, 2^5, \dots, 2^8$
min_samples_leaf	minimale Samples ob Knoten zum Blatt zu wird	5, 10, 20, 40, 80

Tabelle 11: Übersicht der Hyperparameter für Histogram Based Gradient Boosting

Datensatz	Learning Rate	Lossfunktion	Estimatoren	Criterion	Max Leaf Nodes	Min Samples Leaf
1	2^{-3}	logarithmisch	-	-	16	80
2	2^{-5}	logarithmisch	64	Friedman MSE	-	-
3	2^{-4}	logarithmisch	-	-	16	10
4	2^{-3}	exponential	16	Friedman MSE	-	-

Tabelle 12: Beste Hyperparameterkonfigurationen Gradient Boosting

A.3. Zusätzliche Plots, Tabellen

Feature	Wert von coef__
General Health	0.60829
BMI	0.47275
Age	0.44863
High Blood Pressure	0.36010
High Cholesterol	0.28098
Cholesterol Check	0.22531
Sex	0.13167
Heart Disease or Heart Attack	0.06934
Difficulty Walking	0.03432
Stroke	0.03133
Any Healthcare Coverage	0.00731
No Doctor Because of Cost	0.00120
Smoker	0.00000
Physical Activity	−0.01154
Vegetables Consumption	−0.01317
Fruits Consumption	−0.02185
Mental Health	−0.02510
Education	−0.03015
Physical Health	−0.04412
Income	−0.11431
Heavy Alcohol Consumption	−0.15903

Tabelle 13: Coeffizienten Logistic Regression beim Datensatz 1

Diese Tabelle bezieht sich auf den Datensatz 1. Eine genaue Auflistung der Features und deren Bedeutung finden Sie hier [4].

Ergebnisse Testmenge

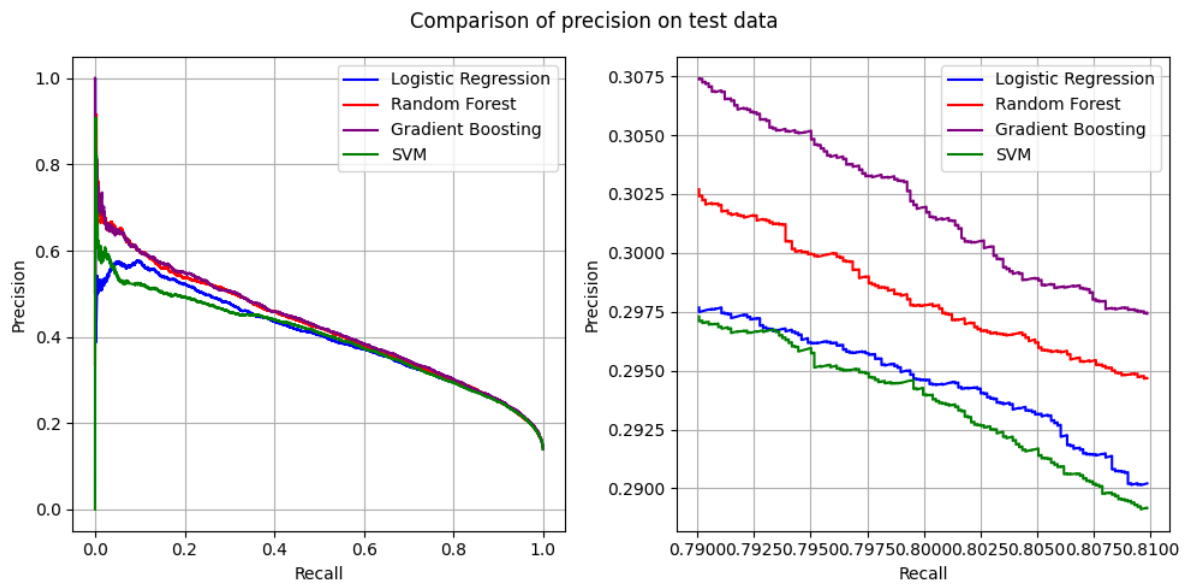


Abbildung 9: Performance Datensatz 1

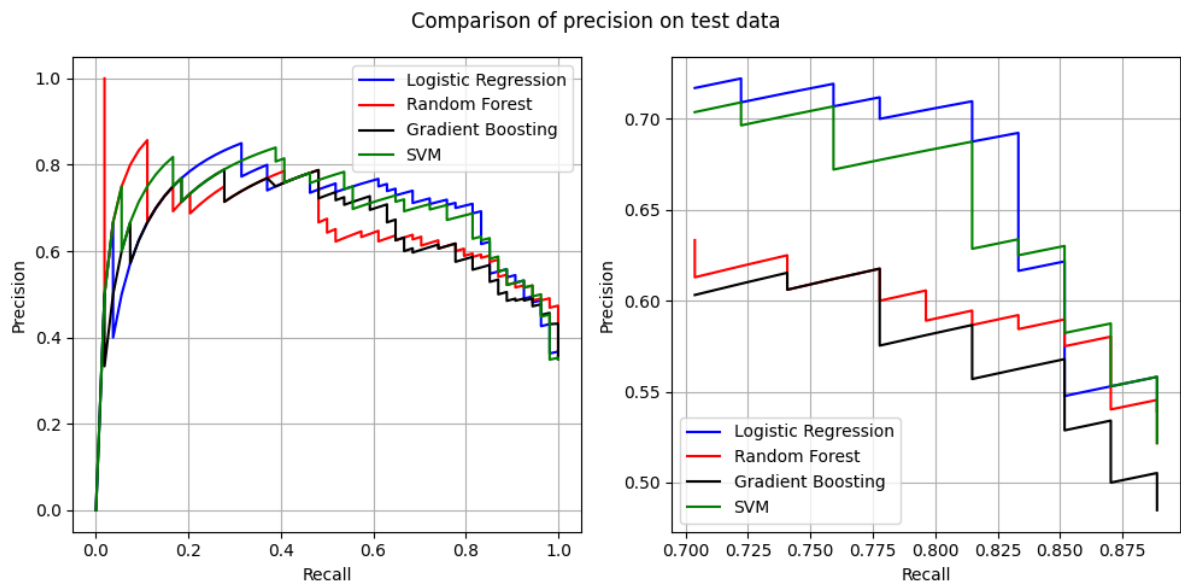


Abbildung 10: Performance Datensatz 2

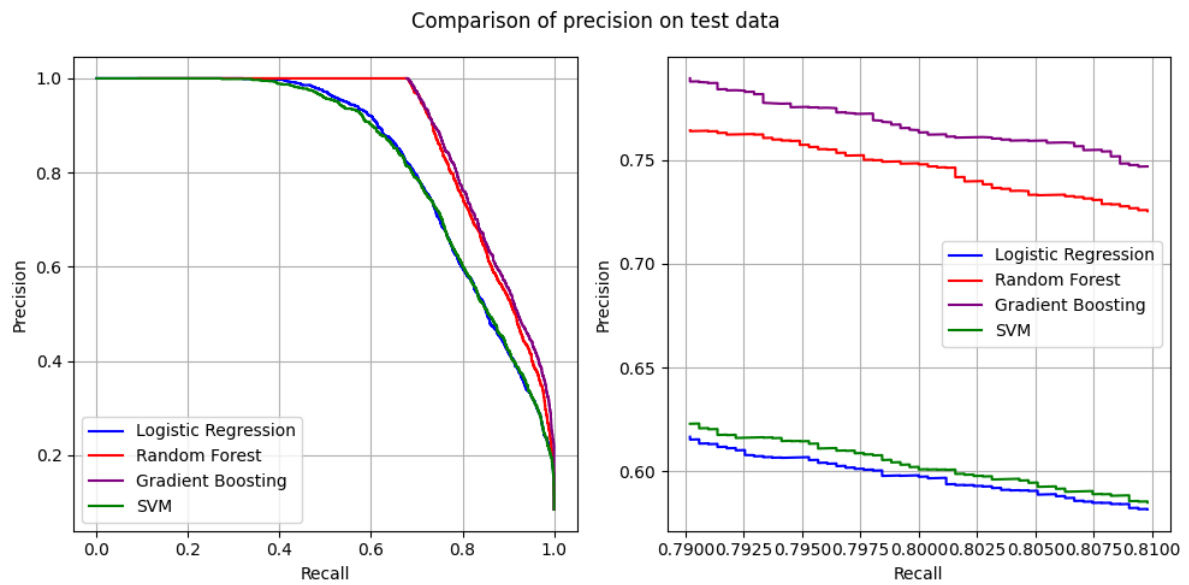


Abbildung 11: Performance Datensatz 3

Ergebnisse Trainingsmenge

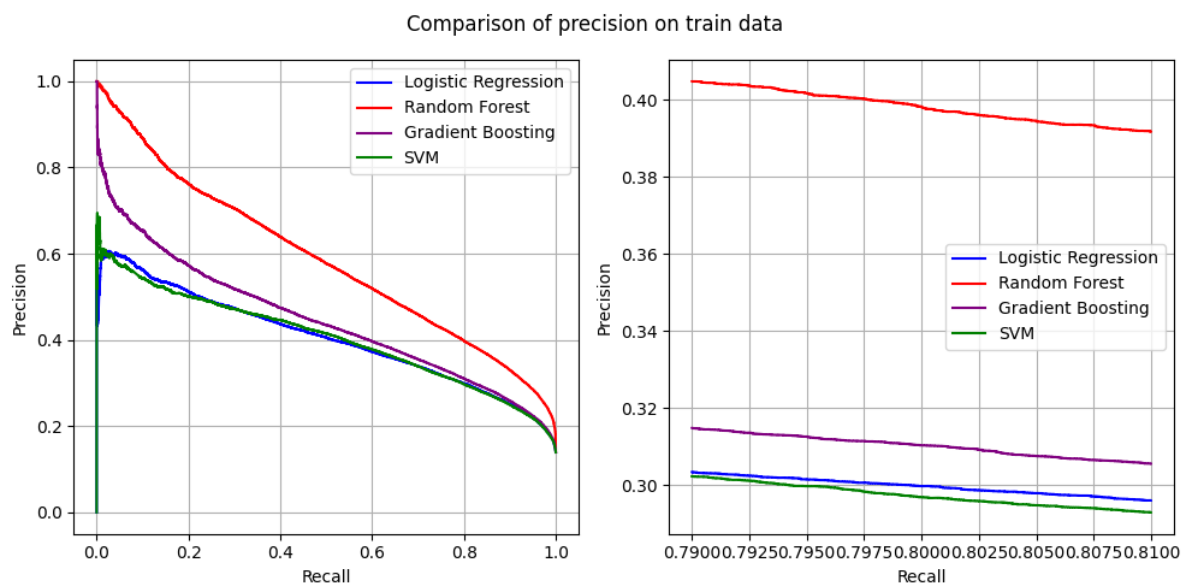


Abbildung 12: Performance Trainingsmenge Datensatz 1

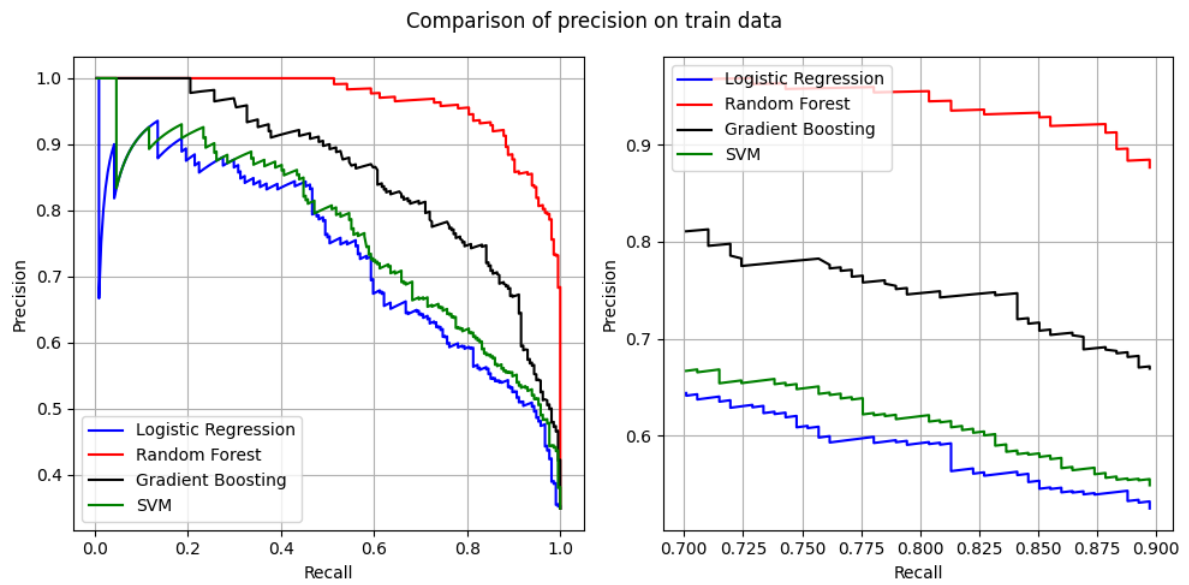


Abbildung 13: Performance Trainingsmenge Datensatz 2

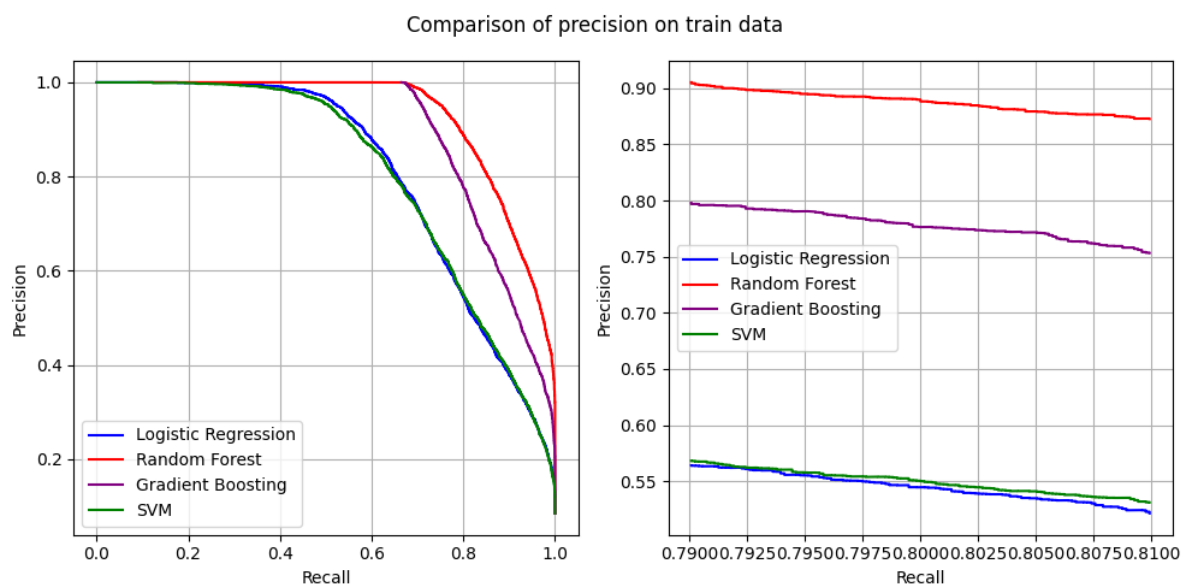


Abbildung 14: Performance Trainingsmenge Datensatz 3

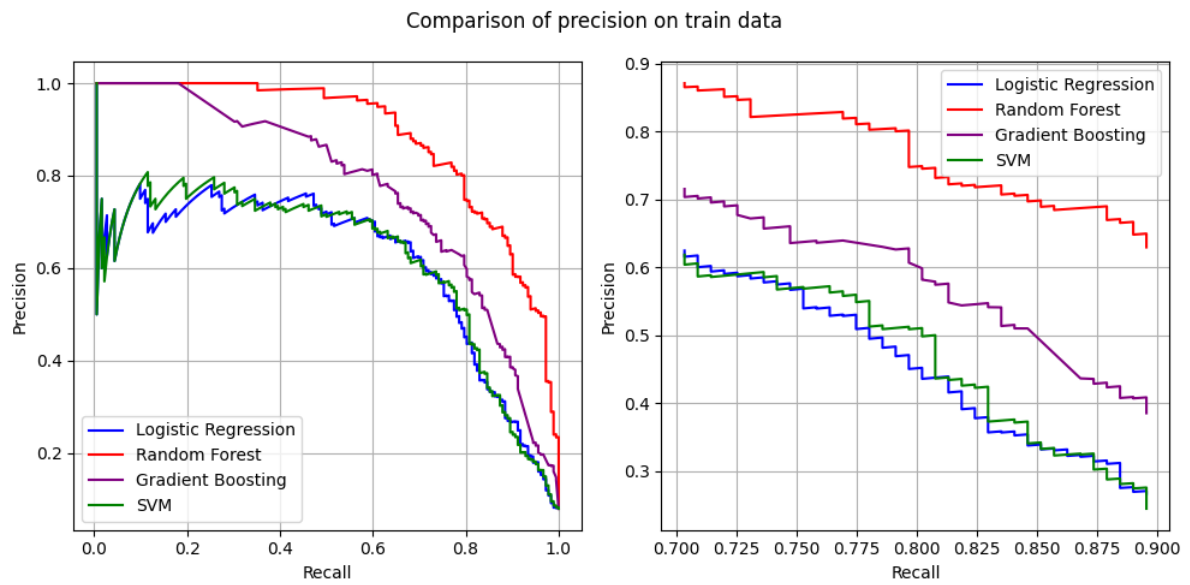


Abbildung 15: Performance Trainingsmenge Datensatz 4